SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I (P.Veeraswami / 192311253) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Veeraswami

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWERS

Answer 1: ATM Withdrawal Module – Black-Box Reverse Engineering

1. Understanding of the System

Only input–output behaviour is available. The following behaviour was observed (assumed from the task description): amounts must be positive multiples of ₹100; the minimum withdrawal is ₹100; a single transaction above ₹20,000 shows 'Limit exceeded'; a withdrawal above the account balance shows 'Insufficient funds'; a non-numeric or zero amount shows 'Invalid amount'. The goal is to infer how the module was designed and tested, then rebuild a better test set.

2. Decomposition

Functional unit	Input	Observed output / rule
F1 Card & PIN check	Card status, PIN	Blocked card / wrong PIN messages
F2 Amount format validation	Entered amount	Reject non-numeric, zero, negative, not multiple of 100
F3 Per-transaction limit check	Amount	Reject above ₹20,000

Functional unit	Input	Observed output / rule
F4 Daily limit check	Amount + today's total	Reject if cumulative total > ₹50,000
F5 Balance validation	Amount, balance	Reject if amount > balance
F6 Cash availability / dispensing	Amount, notes in cassette	'Cannot dispense' if notes are insufficient
F7 Transaction update & receipt	Approved amount	Balance reduced, receipt printed

3. Identification of Components

Equivalence classes and boundaries inferred for the amount field (limit = 20,000, minimum = 100, step = 100, balance = B):

Class	Description	Representative	Expected output	Boundary values
EC1 (invalid)	Amount ≤ 0	-100, 0	Invalid amount	0, 1
EC2 (invalid)	Not a multiple of 100	150	Invalid amount	99, 101
EC3 (valid)	100 ≤ amount ≤ 20,000, multiple of 100, ≤ B	5,000	Success	100, 200, 19,900, 20,000
EC4 (invalid)	Amount > 20,000	25,000	Limit exceeded	20,100
EC5 (invalid)	Amount ≤ 20,000 but > balance	B = 3,000; 5,000	Insufficient funds	B, B + 100
EC6 (invalid)	Non-numeric / blank	'abc'	Invalid amount	–

4. Reconstruction

Inferred technique: the observed messages map one-to-one to input classes, which indicates Equivalence Class Partitioning (one test per message) with, at most, a single boundary test around the 20,000 limit. Evaluation of adequacy: (1) boundaries at 100 and at the balance are not shown to be tested; (2) the daily cumulative limit and cash-availability cases are absent; (3) combinations (amount over the limit and also over the balance – which message has priority?) were not tested; (4) no sequence tests (two withdrawals in a row). Therefore the adequacy is partial: functional coverage is good, boundary and combination coverage is weak.

Reconstructed improved test set:

TC	Balance	Amount	Expected result
1	50,000	0	Invalid amount
2	50,000	-100	Invalid amount
3	50,000	99	Invalid amount
4	50,000	100	Success (minimum boundary)
5	50,000	150	Invalid amount (not multiple of 100)
6	50,000	200	Success (min + 1 step)
7	50,000	19,900	Success
8	50,000	20,000	Success (maximum boundary)
9	50,000	20,100	Limit exceeded
10	3,000	3,000	Success, balance 0 (balance boundary)
11	3,000	3,100	Insufficient funds
12	30,000	25,000	Limit exceeded (limit has priority or as specified)
13	2,000	25,000	Both rules violated – message priority as specified
14	60,000	20,000 ×3 in one day (50,000 daily limit)	Third withdrawal rejected: daily limit
15	50,000	10,000 with only 5,000 in cassette	Cannot dispense; account not debited
16	50,000	'abc' / blank / 1E3	Invalid amount

TC	Balance	Amount	Expected result
17	Blocked card or wrong PIN ×3	1,000	Rejected; card retained after third failure
18	Cash dispensed, power or network lost	5,000	No debit without dispensing (transaction rolled back)

5. Documentation

Summary: observed behaviour → 7 functional units → 6 amount classes with 11 boundary values → 18 test cases. Coverage targets: 100% of equivalence classes and 100% of boundary values for each limit (minimum, maximum, balance, daily). The improved set adds the boundary, combination and sequence cases that the original suite lacked.

Answer 2: Execution Traces and Code Snippets – White-Box Reverse Engineering

1. Understanding of the System

No test documentation is available, so the testing approach is inferred from the code and traces. Assumed code snippet (order total calculation): 1 total = 0; i = 0; 2 while (i < n) { 3 if (qty[i] <= 0) { 4 err++ } else { 5 total += price[i] × qty[i]; 6 if (total > 10000) { 7 total = total × 0.95 } } 8 i++ } 9 return total. Assumed traces: Trace A (n = 3, valid quantities, total below 10000): nodes 1-2-3-5-6-8 (repeated) -2-9. Trace B (n = 1, qty = 0): 1-2-3-4-8-2-9.

2. Decomposition

The module has three logical units: input validation (node 3-4), calculation (nodes 5), discount rule (nodes 6-7) and loop control (nodes 2 and 8). Decision points: D2 'i < n' (loop), D3 'qty ≤ 0', D6 'total > 10000'. Loop: one while loop over the n order lines.

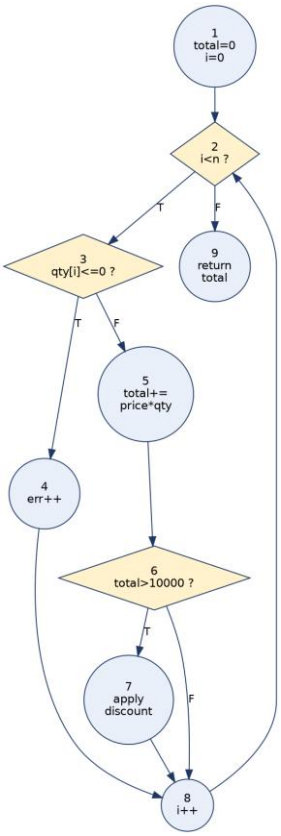


Figure 1: Control Flow Graph of the order-total module

3. Identification of Components

Nodes N = 9, edges E = 11 (1-2, 2-3, 2-9, 3-4, 3-5, 4-8, 5-6, 6-7, 6-8, 7-8, 8-2). Cyclomatic complexity V(G) = E − N + 2 = 4 (also 3 predicates + 1). Independent paths:

Path	Route	Description	Executed by existing traces?
BP1	1-2-9	Empty order (n = 0)	No
BP2	1-2-3-4-8-2-9	Invalid quantity	Yes (Trace B)
BP3	1-2-3-5-6-8-2-9	Valid line, no discount	Yes (Trace A)
BP4	1-2-3-5-6-7-8-2-9	Valid line, discount applied	No

4. Reconstruction

Coverage evaluation: the two traces execute 8 of 9 nodes (statement coverage ≈ 89%; node 7 is missing), 9 of 11 edges (branch coverage ≈ 82%; edges 6-7 and 7-8 are missing) and 2 of 4 basis paths (50%). The tester appears to have used a requirement-driven valid case and one error case, with no coverage criterion beyond 'main flow plus one error'. Because the discount statement was never executed, a defect in it (for example a wrong threshold or wrong factor) could not be found.

TC	Input	Path	Expected result
1	n = 0	BP1	total = 0, no error
2	n = 1, qty = 0	BP2	err = 1, total = 0
3	n = 1, price 100, qty 5	BP3	total = 500
4	n = 1, price 5,000, qty 3	BP4	15,000 × 0.95 = 14,250
5	n = 1, total exactly 10,000	BP3 (boundary)	10,000, no discount
6	n = 1, total 10,001	BP4 (boundary)	Discount applied
7	n = 3: [valid, qty 0, valid large]	Multi-iteration BP3 + BP2 + BP4	Correct running total and error count
8	Loop limits: n = 1, 2, maximum, maximum + 1	Loop boundary	Loop ends correctly; no overflow

5. Documentation

The tests TC 1 to 4 give 100% statement, 100% branch and 100% basis-path coverage; TC 5 to 8 add the threshold boundary and loop-count tests that path coverage alone does not force. Final coverage report: statement 100%, branch 100%, independent paths 4/4, loop counts 0/1/2/max.

Answer 3: Web Application (Login and Transaction) – Technique Reverse Engineering from Defect Logs

1. Understanding of the System

Only defect logs and results exist. Assumed defect log entries: (D1) password with exactly 8 characters rejected although 8 is the minimum; (D2) password of 21 characters accepted though the maximum is 20; (D3) account not locked after the third failed login (locked only after the fourth); (D4) transfer of exactly ₹1,00,000 rejected though it is the limit; (D5) session remains active after logout; (D6) debit made but credit not posted when the destination account is closed (no rollback); (D7) negative amount accepted.

2. Decomposition

Component	Responsibility	Defect IDs
C1 Credential validation	Username/password format and length	D1, D2
C2 Authentication & lockout	Verify credentials, count failures, lock	D3
C3 Session management	Create, expire, destroy session	D5
C4 Transaction input validation	Amount, destination account	D4, D7
C5 Transaction processing	Debit, credit, rollback, audit	D6

3. Identification of Components

Defect-pattern analysis and inferred techniques:

Defect pattern	Inference	Likely technique used	Gap
D1, D2, D4: faults exactly on limits	Tests reached the boundaries, but only some of them	BVA (partial)	Not all limits and not both sides
D7: invalid class accepted	Invalid class was tried	ECP with negative tests	Not applied to every field
D3: counter off by one	A sequence of 3 to 4 failures was run	Branch testing of the lockout decision	Decision tested at one value only
D5, D6: errors in error/exit paths	Found by path/branch testing or in production	Statement/branch coverage on main paths	Rollback and logout paths not covered

Coverage gaps: no systematic BVA on every field, no state/sequence tests for session and lockout, no path tests for transaction failure branches, no condition coverage of compound validations.

4. Reconstruction

Reconstructed integrated strategy: (1) ECP for each field: username (valid, empty, too long, special characters), password (< 8, 8 to 20, > 20), amount (≤ 0, valid, > limit), destination (valid, closed, non-existent). (2) BVA at 7, 8, 9, 19, 20, 21 characters and at amounts 0.01, 1,00,000 and 1,00,000.01. (3) Path testing using the CFG of the login: D1 fields valid? D2 credentials valid? D3 failures ≥ 3? giving V(G) = 4 and four basis paths.

TC	Technique	Input / scenario	Expected result
1	BVA	Password of 7 / 8 / 20 / 21 characters	Reject / accept / accept / reject
2	ECP	Empty username; special characters; SQL string	Rejected with message
3	Path BP1	Empty fields	Validation error, no authentication attempt
4	Path BP2	Valid format, wrong password (1st and 2nd attempt)	Reject; counter increases
5	Path BP3	Third wrong password	Account locked; further correct password refused
6	Path BP4	Correct credentials	Session created; failure counter reset
7	State	Logout, then use old session token / back button	Access denied
8	BVA	Transfer 0.01, 1,00,000, 1,00,000.01, -1	Accept / accept / reject / reject
9	Path (failure branch)	Destination closed after debit	Whole transaction rolled back; audit entry
10	Path (failure branch)	Network failure between debit and credit	Consistent state, no duplicate on retry

5. Documentation

The reconstructed approach justifies each technique through the defect it prevents: BVA for D1, D2 and D4; ECP for D7; path testing for D3, D5 and D6. Coverage targets: 100% classes, 100% boundaries, 100% branches and all basis paths of the login and transaction modules. The logic is traceable from defect pattern to technique to test case.

Answer 4: Legacy Inventory System – Business Logic from Transaction Logs

1. Understanding of the System

Only logs of stock-update transactions are available. Assumed log extract (stock before, operation, result):

Log	Before	Operation	After	Status	Alert
L1	50	Issue 5	45	SUCCESS	–
L2	15	Issue 5	10	SUCCESS	–

Log	Before	Operation	After	Status	Alert
L3	10	Issue 1	9	SUCCESS	LOW_STOCK
L4	9	Issue 9	0	SUCCESS	OUT_OF_STOCK
L5	0	Issue 1	0	FAILED	INSUFFICIENT_STOCK
L6	40	Issue 0	40	FAILED	INVALID_QTY
L7	40	Issue -3	40	FAILED	INVALID_QTY
L8	950	Receive 100	950	FAILED	EXCEEDS_CAPACITY
L9	940	Receive 50	990	SUCCESS	–
L10	990	Receive 10	1000	SUCCESS	–

2. Decomposition

Functional unit	Logic inferred from the logs
U1 Quantity validation	Quantity must be a positive integer (L6, L7 fail)
U2 Issue (stock decrease)	Allowed if quantity ≤ current stock (L4 succeeds at exactly 0 remaining, L5 fails at stock 0)
U3 Receipt (stock increase)	Allowed if stock + quantity ≤ 1000 (L10 succeeds at exactly 1000, L8 fails at 1050)
U4 Low-stock alert	LOW_STOCK when resulting stock < 10 (L2 at 10 gives no alert, L3 at 9 gives an alert)
U5 Out-of-stock alert	OUT_OF_STOCK when resulting stock = 0

3. Identification of Components

Hidden boundaries discovered: reorder threshold between 9 and 10, capacity 1,000, minimum quantity 1, and stock floor 0. Equivalence partitions for issue quantity q at stock s: $q \le 0$ (invalid); $1 \le q \le s - 10$ (success, no alert); $s - 9 \le q \le s - 1$ (success, low-stock alert); $q = s$ (success, out-of-stock); $q > s$ (insufficient). For receipt: $q \le 0$ invalid; $s + q \le 1000$ success; $s + q > 1000$ rejected.

Gaps in the logs (cases never observed): stock exactly at 10 reached by a receipt; alert cleared after replenishment; repeated alert for the same item; receipt reaching 1001; non-integer or very large quantity; two simultaneous updates of one item; negative or inconsistent stock at start.

4. Reconstruction

TC	Stock	Operation	Expected result
1	50	Issue 0	Fail: INVALID_QTY
2	50	Issue 1	Success: 49, no alert
3	20	Issue 10	Success: 10, no alert (threshold boundary)
4	20	Issue 11	Success: 9, LOW_STOCK
5	20	Issue 20	Success: 0, OUT_OF_STOCK
6	20	Issue 21	Fail: INSUFFICIENT_STOCK
7	8 (low alert active)	Receive 2	Success: 10, alert cleared
8	0	Receive 1	Success: 1, out-of-stock cleared, low-stock alert raised
9	999	Receive 1	Success: 1000
10	999	Receive 2	Fail: EXCEEDS_CAPACITY, stock unchanged
11	1000	Receive 1	Fail: EXCEEDS_CAPACITY
12	50	Issue 2.5 / 'abc' / 10^9	Fail: INVALID_QTY
13	9	Issue 1 twice (two users simultaneously)	One succeeds, no negative stock; one alert only

5. Documentation

Traceability: each test is derived from an inferred rule (U1 to U5) and each hidden boundary (0, 1, 9/10, 1000) is tested on both sides. The test design covers every class and boundary and fills the gaps found in the logs, so the legacy behaviour is protected during any later modernisation.

Answer 5: Library Management .exe with CPU/Memory Profile – White-Box Reverse Engineering

1. Understanding of the System

Only a compiled module and a resource profile from a stress test are available. Assumed profile: (a) CPU 40% and a memory step at start-up; (b) CPU 85% for a time that grows with catalogue size during search; (c) short CPU peaks for each issue and return; (d) a CPU burst on the 'fines report' command; (e) memory that keeps rising after repeated issue/return cycles and does not fall back. Spikes in a profile correspond to loops and heavy decision clusters in the code.

2. Decomposition

Profile observation	Likely code structure (inferred)	CFG node
Start-up CPU and memory step	Loop reading all book records into memory	Load loop (D1)
Search time grows with catalogue; CPU 85%	Nested loop: every book × every keyword/field comparison (about O(n·m))	D6, D7, D8
Small peak per issue	Chain of validations: member valid, book available, loan limit	D9, D10, D11
Burst on fines report	Loop over all members and loans with date comparison	D5, D12
Memory rises across cycles	Objects allocated per issue/return and not released (leak) or list that only grows	Issue and return branches

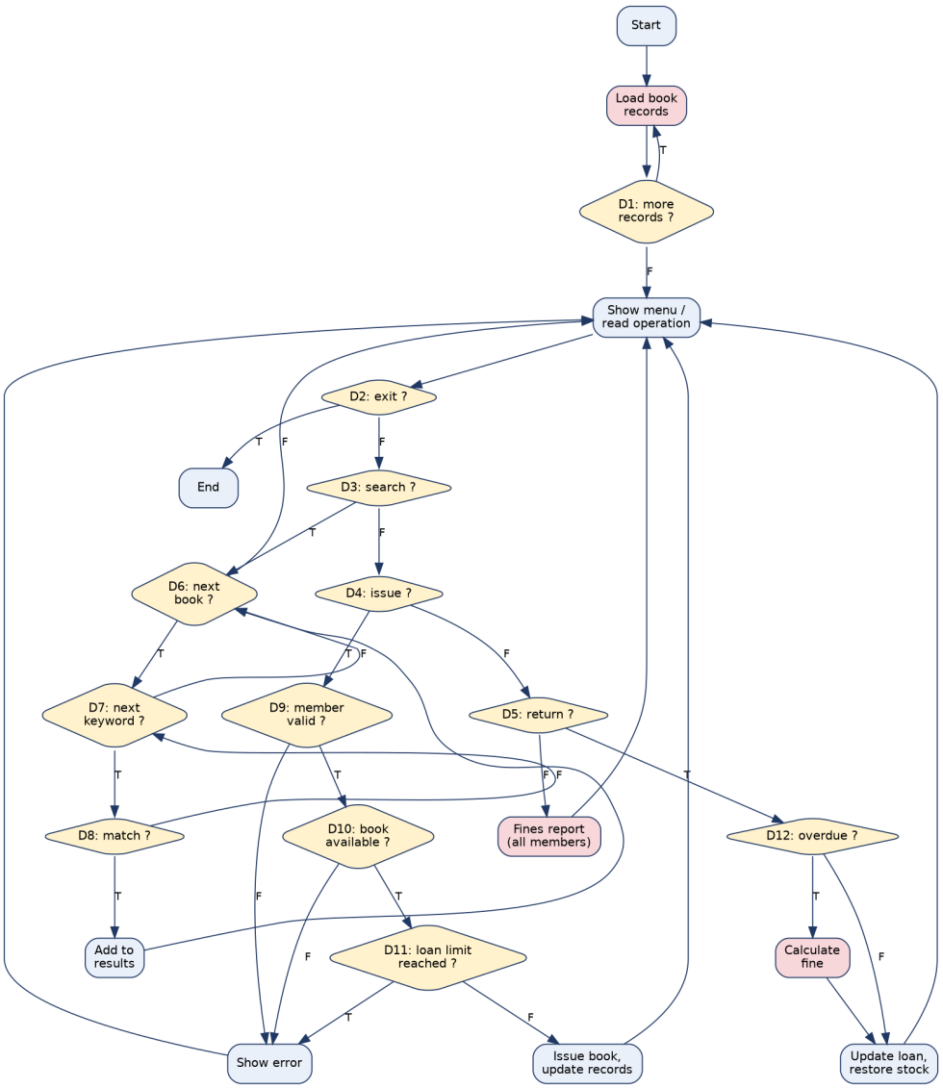


Figure 2: Conceptual Control Flow Graph inferred from the profile (red = resource-intensive nodes)

3. Identification of Components

Decision nodes: D1 more records; D2 exit; D3 search; D4 issue; D5 return; D6 next book; D7 next keyword; D8 match; D9 member valid; D10 book available; D11 loan limit reached; D12 overdue. Twelve predicates give V(G) = 12 + 1 = 13, so at least 13 independent paths exist. Resource-intensive clusters: C1 load loop, C2 search nested loop (D6 to D8), C3 fines report loop, C4 issue/return paths (memory).

4. Reconstruction

TC	Targeted path / cluster	Test data / load	Expected result and measure
1	C1 load loop	0, 1, 10,000 and 1,00,000 records	Start-up time and memory grow linearly
2	C2 search, no match	Largest catalogue, keyword absent	Worst case: all D6/D7 iterations; response within limit
3	C2 search, early match	Keyword matches first book	Loop exits early (fast path)
4	C2 search, many matches	Common keyword, 50% matches	Result list memory bounded; no truncation
5	Issue: D9 false	Invalid member	Error, no resource growth
6	Issue: D10 false	Book already issued	Error
7	Issue: D11 true	Member at loan limit	Error
8	Issue success	Valid issue	Record updated
9	Return on time (D12 false)	Return before due date	No fine
10	Return overdue (D12 true)	Return after due date	Correct fine

TC	Targeted path / cluster	Test data / load	Expected result and measure
11	C3 fines report	1,00,000 members × loans	CPU peak bounded; memory released after report
12	Soak: C4 repeated	10,000 issue/return cycles	Memory returns to baseline (no leak)
13	Menu loop	Many consecutive commands, then exit	Exit works; resources released

Evaluation of impact on stability: the path tests concentrate on the loops and clusters that consume resources, so scalability limits (search time, report time) and the memory leak are measured per path instead of for the system as a whole. Fixes such as indexing the search, paging the fines report and releasing objects after each transaction can be verified by repeating the same path tests and comparing CPU and memory profiles before and after.

5. Documentation

Documentation chain: profile spike → inferred loop or decision cluster → CFG node → path test → measured acceptance threshold (response time, CPU, memory). Coverage target: all 13 independent paths, plus loop boundaries 0/1/n for each resource-intensive loop, plus a soak test to prove stability.

Answer 6: Mobile Payment Gateway – State Machine Reverse Engineering

1. Understanding of the System

From the JSON responses (assumed examples): {"txnId":"T1","status":"PENDING","amount":500}; {"txnId":"T1","status":"SUCCESS","capturedAmount":500};
{"txnId":"T1","status":"REVERSAL","reversedAmount":500,"reason":"USER_REFUND"};
{"txnId":"T2","status":"TIMEOUT","elapsed":31}. Each response is a state of the same transaction identifier, and the fields show the data constraints (guards) for each transition.

2. Decomposition

Component	Function
Transaction creation	Validates amount > 0, unique txnId, currency; creates PENDING
Bank/acquirer interface	Returns confirmation or decline
Timer / watchdog	Moves PENDING to TIMEOUT after 30 s without reply
Settlement	Captures the amount in SUCCESS
Reversal / refund	Reverses captured amount (partial or full), only from SUCCESS or TIMEOUT
Response builder	Returns JSON with status, amounts and reason code

3. Identification of Components

State machine reconstructed from triggers and data constraints:

From	Trigger (event)	Guard (data constraint)	To
(start)	Create payment	amount > 0, txnId unique	PENDING
PENDING	Bank confirms	Reply received within 30 s	SUCCESS
PENDING	Bank declines	Decline code present	FAILED
PENDING	No reply	elapsed > 30 s	TIMEOUT
TIMEOUT	Late confirmation	Bank reply for same txnId	SUCCESS
TIMEOUT	Auto-reversal	No confirmation after review period	REVERSAL
SUCCESS	Refund request	reversedAmount ≤ capturedAmount	REVERSAL
REVERSAL, FAILED	Any event	–	No transition (final states)

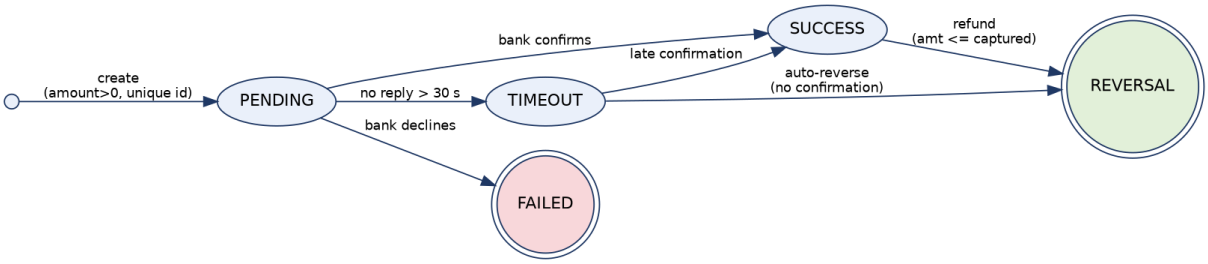


Figure 3: State-transition diagram of the payment gateway

4. Reconstruction

Implicit technique: State Transition Testing with guard-condition (data constraint) checks, and probably 0-switch coverage (each single transition once) because each status appears once in the examples. The reconstructed suite covers all legal transitions, all illegal state–event pairs, guard boundaries and 1-switch sequences.

TC	Type	Scenario	Expected result
L1	Legal	Create → PENDING	Status PENDING, amount echoed
L2	Legal	PENDING → SUCCESS (bank confirm at 29 s)	SUCCESS, capturedAmount = amount
L3	Legal	PENDING → FAILED (decline)	FAILED, reason code
L4	Legal	PENDING → TIMEOUT (no reply, 31 s)	TIMEOUT
L5	Legal	TIMEOUT → SUCCESS (late confirmation)	SUCCESS, no double debit
L6	Legal	TIMEOUT → REVERSAL (auto)	REVERSAL, customer refunded
L7	Legal	SUCCESS → REVERSAL (full refund)	reversedAmount = capturedAmount
L8	Legal	SUCCESS → partial refund, then second partial refund	Total reversed ≤ captured
B1	Boundary	Timeout at 30 s and 30.01 s	PENDING remains / TIMEOUT
B2	Boundary	Refund = captured, captured + 0.01	Accepted / rejected
B3	Boundary	Amount 0, -1, 0.01	Rejected / rejected / accepted
I1	Illegal	SUCCESS → PENDING	Rejected, status unchanged
I2	Illegal	SUCCESS → TIMEOUT	Rejected
I3	Illegal	REVERSAL → SUCCESS	Rejected (final state)
I4	Illegal	FAILED → SUCCESS	Rejected
I5	Illegal	Refund while PENDING	Rejected
I6	Illegal	Duplicate txnId on create	Rejected, no new state
S1	Sequence	Confirm and timeout arriving together (race)	Single consistent final state
S2	Sequence	Duplicate confirmation message	Idempotent, no double capture

5. Documentation

States: 5; legal transitions: 7; illegal state–event pairs identified: all remaining pairs (for example 5 states × 4 events minus the 7 legal ones). The suite covers 100% of states, 100% of legal transitions, all illegal transitions listed above, guard boundaries and race conditions, which are the usual sources of money-handling defects.

Answer 7: Encrypted Communication Module – Fuzzing and BVA

1. Understanding of the System

The module exposes only inputs and encrypted outputs. Fields (assumed): message (text, max 1,024 bytes), key length (128, 192 or 256 bits), recipient ID (alphanumeric, max 32 characters) and mode (CBC or GCM). Fuzzing observations (assumed): message of 1,024 bytes is encrypted and 1,025 bytes returns ERR_LENGTH; a 4,096-byte message makes the service crash; key length 255 and 257 are rejected; recipient ID with a quotation mark returns a database error, and an empty message returns an encrypted block of random-looking data instead of an error.

2. Decomposition

Component	Responsibility	Fuzzing evidence
Data validation layer	Length, charset, null/empty, format checks	Limit at 1,024 works, but 4,096 bypasses the check (validation after copy into a fixed buffer)
Recipient lookup	Finds key/recipient in a database	Special character causes SQL error (injection risk)
Encryption engine	Key setup, IV, padding, cipher	Empty message encrypted (no minimum length rule)
Output encoder	Base64 / hex of ciphertext	No error observed

3. Identification of Components

Error-handling boundaries inferred: message length 0 (accepted incorrectly), 1 to 1,024 (valid), 1,025 and above (rejected, except very large inputs which crash); key lengths exactly 128, 192, 256; recipient 1 to 32 alphanumeric characters.

4. Reconstruction

TC	Field	Value	Expected result
1	Message	0 bytes	Rejected (or defined behaviour)
2	Message	1 byte	Encrypted
3	Message	1,023 / 1,024 bytes	Encrypted
4	Message	1,025 bytes	ERR_LENGTH, no processing
5	Message	4,096 bytes; 1 MB; 100 MB	ERR_LENGTH, no crash, memory stable
6	Message	Multi-byte Unicode of 1,024 characters (bytes > 1,024)	Length counted in bytes; rejected
7	Message	Null byte, control characters, '<script>'	Handled as data, encrypted or rejected, never executed
8	Key length	127, 128, 129	Reject, accept, reject
9	Key length	191, 192, 193	Reject, accept, reject
10	Key length	255, 256, 257, 0, -1	Reject, accept, reject, reject, reject
11	Recipient	0, 1, 32, 33 characters	Reject, accept, accept, reject
12	Recipient	' OR '1'='1 ; '; DROP TABLE users;--	Rejected by whitelist; parameterised query; no DB error
13	Recipient	../../etc/passwd; %00; very long string	Rejected
14	Output	Same message encrypted twice	Different ciphertext (random IV); decrypts correctly

5. Documentation

How the design prevents vulnerabilities: BVA at 1,024/1,025 bytes plus very large inputs verifies that length checks happen before any copy into a fixed-size buffer, which prevents buffer overflow. Whitelist validation and parameterised queries tested by cases 12 and 13 prevent SQL and command injection. Together with fuzzing in the regression suite (random and mutated inputs), failures are found before release. Expected result for all invalid inputs: clear error, no crash, no leak of internal information, and no encryption performed.

Answer 8: Insurance Premium Calculator – 100% Statement Coverage but Missed Defects

1. Understanding of the System

Assumed code: 1 premium = base; 2 if (age < 25 AND hasAccident) premium × 1.5; 4 if (vehicleAge > 10 OR mileage > 100000) premium × 1.2; 6 if (noClaimYears ≥ 5 AND NOT hasAccident) premium × 0.9; 8 return premium. The legacy tests report 100% statement coverage, which can be achieved with only two tests (all

conditions true, then the discount case), so the original approach was statement-coverage testing with a few typical inputs.

2. Decomposition

Unit	Decision	Atomic conditions
Young-driver loading	D1	A: age < 25; B: hasAccident
Vehicle-risk loading	D2	C: vehicleAge > 10; D: mileage > 100,000
No-claim discount	D3	E: noClaimYears ≥ 5; NOT B

3. Identification of Components

Inadequacy of statement coverage: it checks that each statement is executed, but not that each decision takes both outcomes or that each condition in a compound expression influences the outcome. A wrong operator (AND written for OR) or a wrong constant in one condition is not exposed by tests that make the whole decision true once.

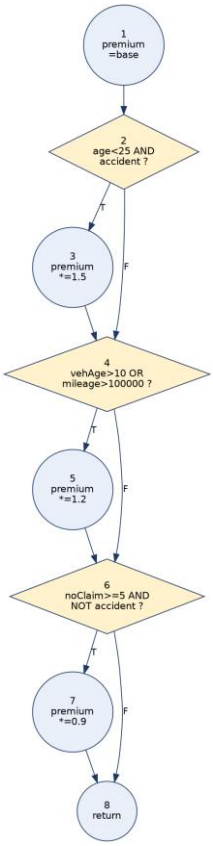


Figure 4: Control Flow Graph of the premium calculator

Nodes N = 8, edges E = 10 (1-2, 2-3, 2-4, 3-4, 4-5, 4-6, 5-6, 6-7, 6-8, 7-8). V(G) = 10 − 8 + 2 = 4, so four basis paths. Three sequential decisions give $2^3 = 8$ possible paths. Important finding: when D1 is true, hasAccident is true, so D3 (needs NOT hasAccident) must be false; therefore the paths (D1 true, D3 true) are infeasible and 6 feasible paths remain.

4. Reconstruction

Strategy: all-path testing of the 6 feasible paths (this also gives 100% branch coverage), then condition-level tests for each compound decision. Base premium = 1,000.

TC	D1	D2	D3	Input (age, accident, vehicle age, mileage, no-claim years)	Expected premium
P1	F	F	F	30, yes, 5, 50,000, 2	1,000
P2	F	F	T	30, no, 5, 50,000, 6	900
P3	F	T	F	30, no, 12, 50,000, 2	1,200
P4	F	T	T	30, no, 12, 50,000, 6	1,080

TC	D1	D2	D3	Input (age, accident, vehicle age, mileage, no-claim years)	Expected premium
P5	T	F	F	22, yes, 5, 50,000, 0	1,500
P6	T	T	F	22, yes, 5, 1,20,000, 0	1,800
–	T	F	T	Infeasible (accident cannot be both true and false)	–
–	T	T	T	Infeasible	–

TC	Targets (condition coverage)	Input	Expected premium
C1	A true, B false (D1 false because B)	22, no, 5, 50,000, 2	1,000
C2	A false, B true (D1 false because A)	30, yes, 5, 50,000, 2	1,000 (same as P1)
C3	C false, D true (mileage only)	30, no, 5, 1,20,000, 2	1,200
C4	C true, D false (vehicle age only)	30, no, 12, 50,000, 2	1,200 (as P3)
C5	E true, B true (D3 false because of accident)	30, yes, 5, 50,000, 6	1,000
C6	Boundaries: age 24 / 25; vehicle age 10 / 11; mileage 100,000 / 100,001; no-claim 4 / 5	each with others at base values	Loading or discount applied exactly at the stated limit

5. Documentation

Coverage achieved: statement 100%, branch 100% (every decision true and false), path 100% of feasible paths (6 of 6), and condition coverage with boundary values. Why the risk is reduced: every decision outcome and the order in which the surcharges are applied (the multipliers are not commutative with rounding) are checked against calculated expected premiums, each atomic condition is shown to influence the result, and the boundary tests detect off-by-one errors in age, mileage and years. Calculation errors therefore cannot hide behind a single true/true test.

Answer 9: Cloud File Synchronisation Tool – Lost-File Defects

1. Understanding of the System

Only user activity logs and 'lost file' reports are available. Assumed patterns: (a) files larger than 2 GB (2,147,483,648 bytes) disappear after sync; (b) 0-byte files are skipped and later deleted from the other device; (c) files with extensions .tmp and .lock, and hidden files, are silently ignored; (d) at bandwidth below 256 kbps, uploads of files above 100 MB time out, the local copy is replaced by an incomplete copy; (e) many files modified at the same time lose the older one (conflict).

2. Decomposition

Component	Function	Suspected defect
Scanner	Detects changed files, applies ignore rules	(b), (c)
Chunker / uploader	Splits files, transfers with retry	(a) size overflow, (d) timeout handling
Network monitor	Chooses chunk size and timeout by speed	(d)
Integrity & commit	Verifies checksum, marks synced, deletes temporary copy	(d) deletes before verifying
Conflict resolver	Handles simultaneous edits	(e)

3. Identification of Components

Inferred ECP of the original developers (and its flaws):

Parameter	Inferred partitions	Observed failure
File size	Small (< 1 MB) – single upload; medium (1 MB to 2 GB) – chunked; large (> 2 GB) – assumed same as medium	Large class not handled (signed 32-bit size overflow at $2^{31} - 1$)
File type	Normal files; temporary/lock files (ignored)	Zero-byte and hidden files fall in no class

Parameter	Inferred partitions	Observed failure
Network speed	Fast (≥ 5 Mbps); normal; no slow class	Slow and interrupted connections not tested

4. Reconstruction

Revised strategy: ECP for each parameter, BVA at the partition limits, then stress and fault-injection tests.

TC	Type	Test	Expected result
1	BVA size	0 byte, 1 byte	Synced and present on all devices
2	BVA size	1 MB − 1, 1 MB, 1 MB + 1	Same result at each side of single/chunked limit
3	BVA size	$2^{31} - 1$, 2^{31}, $2^{31} + 1$ bytes (2 GB boundary)	No data loss; checksum equal
4	BVA size	4 GB − 1, 4 GB, 4 GB + 1; maximum size allowed	Synced or clear rejection, never silent loss
5	ECP type	Hidden file, .tmp, .lock, symbolic link, very long file name, Unicode name	Ignore rule applied as specified and logged
6	ECP network	Throttle to 64 / 256 / 1,024 kbps with a 500 MB file	Resumable upload; original untouched until verified
7	Stress	Drop the network at 50% and 99% of upload; reconnect	Resume from last chunk; no partial file replaces the original
8	Stress	1,00,000 small files in one folder	All synced; time and memory bounded
9	Stress	Concurrent edits of one file on two devices	Both versions kept (conflict copy)
10	Stress	Low disk space on target; quota full	Error shown; no deletion at source
11	Soak	24-hour continuous sync with random changes	No lost file, checksum audit zero mismatch
12	Recovery	Kill the client during commit	Consistent state after restart

5. Documentation

How data loss is prevented: tests 3 and 4 remove the size-overflow defect; tests 6, 7 and 10 prove that the source file is never deleted until the destination checksum is verified; tests 8 to 12 expose concurrency and resource defects that appear only under load. The principle documented for developers is 'verify, then delete', and the suite is rerun for every release together with a checksum audit comparing all files on all devices.

Answer 10: E-Commerce Search and Filter – Decision Table from SQL Queries

1. Understanding of the System

Only the generated SQL is available. Assumed observed queries: Q1: WHERE category = 'Shoes' AND price BETWEEN 1000 AND 3000. Q2: WHERE category = 'Shoes' AND brand IN ('Nike','Adidas'). Q3: WHERE category = 'Shoes' AND in_stock = 1. Q4: WHERE category = 'Shoes' AND rating >= 4 AND brand IN ('Puma'). Q5: WHERE category = 'Shoes' AND brand IN ('Nike','Adidas') AND price BETWEEN 1000 AND 3000 AND in_stock = 1 AND NOT discontinued = 1.

2. Decomposition

Filter	Parameter	Logic inferred
Category	One value, mandatory	Always AND
Brand	Multi-select	OR inside the group (IN), AND with other filters
Price	Min–max range	BETWEEN, inclusive limits, AND
Stock	Checkbox 'in stock only'	AND in_stock = 1; omitted when unchecked
Rating	Minimum stars	AND rating ≥ r
Discontinued	Implicit	NOT discontinued, always applied

3. Identification of Components

Decision logic: result = category AND (brand1 OR brand2 …) AND (price in [min, max]) AND in_stock AND (rating ≥ r) AND NOT discontinued, where each optional filter is simply left out when not selected. Four optional filters (brand B, price P, stock S, rating R) give $2^4 = 16$ rules.

4. Reconstruction

Rule	Brand	Price	Stock	Rating	Expected WHERE clause
R1	Y	Y	Y	Y	category AND brand IN (…) AND price BETWEEN AND in_stock=1 AND rating≥r AND NOT discontinued
R2	Y	Y	Y	N	category AND brand IN (…) AND price BETWEEN AND in_stock=1 AND NOT discontinued
R3	Y	Y	N	Y	category AND brand IN (…) AND price BETWEEN AND rating≥r AND NOT discontinued
R4	Y	Y	N	N	category AND brand IN (…) AND price BETWEEN AND NOT discontinued
R5	Y	N	Y	Y	category AND brand IN (…) AND in_stock=1 AND rating≥r AND NOT discontinued
R6	Y	N	Y	N	category AND brand IN (…) AND in_stock=1 AND NOT discontinued
R7	Y	N	N	Y	category AND brand IN (…) AND rating≥r AND NOT discontinued
R8	Y	N	N	N	category AND brand IN (…) AND NOT discontinued
R9	N	Y	Y	Y	category AND price BETWEEN AND in_stock=1 AND rating≥r AND NOT discontinued
R10	N	Y	Y	N	category AND price BETWEEN AND in_stock=1 AND NOT discontinued
R11	N	Y	N	Y	category AND price BETWEEN AND rating≥r AND NOT discontinued
R12	N	Y	N	N	category AND price BETWEEN AND NOT discontinued
R13	N	N	Y	Y	category AND in_stock=1 AND rating≥r AND NOT discontinued
R14	N	N	Y	N	category AND in_stock=1 AND NOT discontinued
R15	N	N	N	Y	category AND rating≥r AND NOT discontinued
R16	N	N	N	N	category AND NOT discontinued

Additional tests beyond the 16 rules (the cases the table does not capture):

TC	Case	Expected result
A1	Two or more brands selected	IN list with OR; products of all selected brands
A2	Price min = max	Only items with exactly that price (inclusive limits)
A3	Price min > max	Validation error or swapped values as specified; not an empty page
A4	Price min = 0 / negative; very large maximum	Handled; no SQL error
A5	Filter combination returns no products	Empty-state message, no error
A6	Discontinued item matching all other filters	Never shown
A7	Keyword containing quote or SQL fragment	Treated as text (parameterised); no injection
A8	Clear all filters	Returns the category list as in R16

Completeness evaluation: the model covers every combination of the four optional filters (16 of 16 rules), the inclusive boundaries of the price range and the fixed rules (category and NOT discontinued). It does not cover sorting, pagination, text search and the number of selected values inside a group, which are listed as separate test items. Residual risk is therefore low for filter logic and must be handled with separate models for the excluded features. If a new filter is added, the table doubles ($2^5 = 32$ rules), at which point pairwise selection should be applied to keep the suite manageable.

5. Documentation

Documentation chain: observed SQL → filter logic (AND between groups, OR inside brand group, NOT for discontinued) → decision table of 16 rules → 8 supplementary tests. Each test is traceable to a rule or boundary, and expected results are expressed as the exact WHERE clause and the resulting product set.